



SRI RAMACHANDRA

INSTITUTE OF HIGHER EDUCATION AND RESEARCH

(Category - I Deemed to be University) Porur, Chennai

AI-BASED SMART INTERVIEW SYSTEM

PROJECT REPORT

Submitted by

NARESH KUMAR B - E0322008

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence and Data Analytics)

Sri Ramachandra Faculty of Engineering and Technology

Sri Ramachandra Institute of Higher Education and Research, Porur, Chennai - 600116

MAY 2026

ABSTRACT

The AI-Based Smart Interview System is designed to automate the first-level interview assessment process for candidates. Traditional interview screening requires more time, manual effort, and evaluator consistency, especially when many candidates apply for similar roles. The proposed system solves this problem by providing a webbased smart interview platform that conducts structured interviews, evaluates answers using Natural Language Processing techniques, monitors basic candidate presence through computer vision, stores interview details, and generates a professional report.

The system is implemented using Python Flask as the backend framework. Candidate authentication is handled through OTP verification, after which candidates can select roles such as Manual Testing, Automation Testing, AI Tech Support, or Software Development. The interview contains role-based aptitude and programming questions. Candidate answers are evaluated using exact matching, keyword matching, TF-IDF cosine similarity, and communication-quality heuristics. OpenCV is used for webcam-based face detection, smile detection, multiple-face alerting, and confidenceindex calculation. SQLite stores user data, roles, questions, interviews, answers, scores, shortlist status, and report paths.

The final system generates PDF reports containing overall score, question-wise feedback, proctoring alerts, confidence index, and shortlist decision. This project demonstrates a complete AI-enabled interview workflow suitable for academic review and future extension into a larger recruitment-support application.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	SECTIONS	PAGE NO.
1	INTRODUCTION	1.1 OBJECTIVE 1.2 PROBLEM STATEMENT	4

2	LITERATURE REVIEW	2.1 SUMMARY TABLE 2.2 SUMMARY OF LITERATURE SURVEY	5
3	METHODOLOGY	3.1 CANDIDATE REGISTRATION 3.2 ROLE SELECTION 3.3 QUESTION GENERATION 3.4 ANSWER EVALUATION 3.5 FACE ANALYSIS 3.6 REPORT GENERATION	9
4	MODEL ARCHITECTURE	4.1 SYSTEM ARCHITECTURE OVERVIEW 4.2 NLP AND COMPUTER VISION ARCHITECTURE 4.3 ARCHITECTURAL WORKFLOW	10

CHAPTER 1 INTRODUCTION

Interview assessment is an important process for selecting candidates based on technical knowledge, communication ability, reasoning skill, and professional behaviour. In manual interviews, evaluators ask questions, listen to candidate responses, make notes, and later compare performance. This process becomes difficult when many candidates must be evaluated in a limited time. A smart interview system can support evaluators by maintaining a consistent question pattern, applying measurable scoring logic, and generating structured reports.

This project develops an AI-Based Smart Interview System using the wittmann_interview_ai workflow. The system is inspired by company-specific interview preparation and supports a complete online assessment flow. Candidates enter their name, email, and phone number, verify OTP, select a role, answer interview questions, and submit the interview. The backend evaluates answers and generates reports for the administrator.

The system uses local AI techniques instead of depending entirely on paid external APIs. NLP scoring is performed using TF-IDF cosine similarity, keyword matching, and communication

heuristics. Computer vision support is provided through OpenCV Haar cascade models that detect face presence and multiple faces during the interview. The project therefore combines web development, NLP, computer vision, database management, and report generation in one working application.

1.1 OBJECTIVE

- To develop an intelligent system that conducts role-based smart interviews.
- To verify candidates using OTP before the interview begins.
- To analyze candidate answers using NLP-based scoring techniques.
- To monitor basic face presence and confidence signals using OpenCV.
- To store candidate, question, answer, score, and report details in SQLite.
- To generate a structured interview report with score, feedback, and shortlist status.

1.2 PROBLEM STATEMENT

Manual interview screening is time-consuming, inconsistent, and difficult to scale. Candidates may not receive structured feedback, and evaluators may find it difficult to compare answers uniformly. The proposed project solves this problem by creating a web-based AI interview system that collects candidate details, conducts role-specific interviews, evaluates answers automatically, records proctoring signals, and generates a report for review.

CHAPTER 2 LITERATURE REVIEW

2.1 SUMMARY TABLE

No	Paper Title	Authors	Year	Method Used	Contribution
1	Automated Prediction and Analysis of Job Interview Performance	I. Naim et al.	2015	Multimodal ML	Predicts interview performance using verbal and nonverbal cues.
2	Hirability in the Wild: Analysis of Online Video Resumes	L. Nguyen, D. Gatica-Perez	2016	Video Analytics	Analyzes online video resumes for hirability impressions.

3	Automated Video Interview Judgment on Large Corpus	L. Chen et al.	2017	Speech + Vision	Uses speech, facial, and language features for interview judgment.
4	Words Worth: Verbal Content and Hirability Impressions	S. Muralidhar et al.	2018	NLP	Shows importance of spoken content in candidate assessment.
5	HireNet: Hierarchical Attention for Video Job Interviews	L. Hemamou et al.	2019	Deep Learning	Models interview questionanswer sequences using attention.
6	Slices of Attention in Asynchronous Job Interviews	L. Hemamou et al.	2019	Attention Model	Improves interpretability of automated interview predictions.
7	Automatic Scoring of Answer Transcriptions	Z. Liu et al.	2020	Graph Neural Network	Scores interview answers from transcripts using
					semantic reasoning.
8	Mitigating Bias in Algorithmic Hiring	M. Raghavan et al.	2020	AI Fairness	Discusses fairness and audit risks in hiring algorithms.
9	Scikit-learn: Machine Learning in Python	F. Pedregosa et al.	2011	ML Library	Provides practical tools for TF-IDF and similarity computation.

10	A Statistical Interpretation of Term Specificity	K. Sparck Jones	1972	TF-IDF	Introduces the foundation of termweighting for text relevance.
11	BERT: Pre-training of Deep Bidirectional Transformers	J. Devlin et al.	2019	Transformer NLP	Provides future scope for better semantic answer scoring.
12	The OpenCV Library	G. Bradski	2000	Computer Vision	Supports realtime image processing and face detection.
13	Robots are Judging Me	A. Koechling et al.	2022	Recruitment Fairness	Studies applicant fairness perception of AI recruitment tools.
14	Artificial Intelligence in Personnel Selection	H. Suen et al.	2019	HR Analytics	Reviews AI usage in selection and interview assessment.
15	Algorithmic Recruitment and Selection Systems	N. Guenole et al.	2022	Review Study	Explains validity and governance needs in AI selection systems.

2.2 SUMMARY OF LITERATURE SURVEY

I. Naim et al. (2015) proposed or discussed automated prediction and analysis of job interview performance. The work is useful for this project because it supports the idea of predicts interview performance using verbal and nonverbal cues.

L. Nguyen, D. Gatica-Perez (2016) proposed or discussed hirability in the wild: analysis of online video resumes. The work is useful for this project because it supports the idea of analyzes online video resumes for hirability impressions.

L. Chen et al. (2017) proposed or discussed automated video interview judgment on large corpus. The work is useful for this project because it supports the idea of uses speech, facial, and language features for interview judgment.

S. Muralidhar et al. (2018) proposed or discussed words worth: verbal content and hirability impressions. The work is useful for this project because it supports the idea of shows importance of spoken content in candidate assessment.

L. Hemamou et al. (2019) proposed or discussed hirenet: hierarchical attention for video job interviews. The work is useful for this project because it supports the idea of models interview question-answer sequences using attention.

L. Hemamou et al. (2019) proposed or discussed slices of attention in asynchronous job interviews. The work is useful for this project because it supports the idea of improves interpretability of automated interview predictions.

Z. Liu et al. (2020) proposed or discussed automatic scoring of answer transcriptions. The work is useful for this project because it supports the idea of scores interview answers from transcripts using semantic reasoning.

M. Raghavan et al. (2020) proposed or discussed mitigating bias in algorithmic hiring. The work is useful for this project because it supports the idea of discusses fairness and audit risks in hiring algorithms.

F. Pedregosa et al. (2011) proposed or discussed scikit-learn: machine learning in python. The work is useful for this project because it supports the idea of provides practical tools for tf-idf and similarity computation.

K. Sparck Jones (1972) proposed or discussed a statistical interpretation of term specificity. The work is useful for this project because it supports the idea of introduces the foundation of termweighting for text relevance.

J. Devlin et al. (2019) proposed or discussed bert: pre-training of deep bidirectional transformers. The work is useful for this project because it supports the idea of provides future scope for better semantic answer scoring.

G. Bradski (2000) proposed or discussed the opencv library. The work is useful for this project because it supports the idea of supports real-time image processing and face detection.

A. Koechling et al. (2022) proposed or discussed robots are judging me. The work is useful for this project because it supports the idea of studies applicant fairness perception of ai recruitment tools.

H. Suen et al. (2019) proposed or discussed artificial intelligence in personnel selection. The work is useful for this project because it supports the idea of reviews ai usage in selection and interview assessment.

N. Guenole et al. (2022) proposed or discussed algorithmic recruitment and selection systems. The work is useful for this project because it supports the idea of explains validity and governance needs in ai selection systems.

CHAPTER 3 METHODOLOGY

The methodology follows a structured pipeline beginning with candidate registration and ending with report generation. Each module is implemented separately so that authentication, role selection, question management, answer scoring, face analysis, database operations, and report generation can work together as one complete system.

3.1 Candidate Registration

The candidate enters name, email, and phone number on the home page. The system generates a six-digit OTP and verifies the candidate before allowing access to the interview flow.

3.2 Role Selection

After OTP verification, the candidate selects a role. The available roles are Manual Testing, Automation Testing, AI Tech Support, and Software Development. The selected role controls the question pattern.

3.3 Question Generation

Questions are generated from stored role patterns. Each role contains aptitude topics and programming topics. The system can generate questions locally and can optionally use an AI provider if configured.

3.4 Answer Evaluation

Objective questions are evaluated using exact-answer matching. Descriptive answers are evaluated using keyword matching, TF-IDF cosine similarity between the candidate answer and ideal answer, and a communication score based on length and vocabulary variety.

3.5 Face Analysis

During the interview, JavaScript captures webcam frames and sends them to the Flask backend. OpenCV detects face presence, multiple faces, and smile signals. These values are converted into a confidence index and proctoring alerts.

3.6 Report Generation

After submission, the system calculates the overall score, shortlist status, and proctoring summary. The ReportLab library generates a PDF report, and the database stores the report path for admin download.

CHAPTER 4 MODEL ARCHITECTURE

The AI-Based Smart Interview System is designed as a multi-layer application. The presentation layer handles candidate interaction, the application layer handles routes and APIs, the AI layer performs NLP and computer vision analysis, and the data/report layer stores and presents final results.

4.1 System Architecture Overview

Layer	Components and Responsibilities
Presentation Layer	HTML, CSS, JavaScript, webcam capture, speech-to-text, role pages, interview page.
Application Layer	Flask routes for registration, OTP, role selection, interview, admin database, reports, and API submission.
AI Evaluation Layer	Keyword matching, exact answer scoring, TFIDF cosine similarity, communication score, and feedback generation.
Computer Vision Layer	OpenCV face detection, smile detection, multiple-face alert, and confidence-index calculation.
Data / Report Layer	SQLite database, candidate answers, interview scores, shortlist decision, and PDF report generation.

4.2 NLP and Computer Vision Architecture

- NLP Input: Candidate answer and ideal answer.
- NLP Process: Text cleaning, keyword checking, TF-IDF vectorization, cosine similarity, and total-score calculation.
- NLP Output: Question-wise score, matched keywords, missing keywords, and feedback.
- CV Input: Base64 webcam frame from browser.
- CV Process: Decode image, convert to grayscale, detect face and smile using Haar cascades.
- CV Output: Face status, multiple-face alert, confidence hint, and confidence index.

4.3 Architectural Workflow

	OTP	Role	Interview	AI Score	Shortlist	Report
--	-----	------	-----------	----------	-----------	--------

The workflow begins with candidate registration and OTP verification. Once verified, the candidate selects a role and attends the interview. The submitted answers and webcam signals are processed by AI modules. The final score and proctoring summary are used to create a shortlist decision and generate the report.

4.4 Detailed Design for Proposed System

The detailed design of the proposed AI-Based Smart Interview System is divided into candidate interaction, backend processing, AI evaluation, database storage, and report generation. The candidate first registers and verifies OTP. After verification, the candidate selects a role and attends the interview. The Flask backend assigns a role-based question paper and stores the exact assigned questions so that later question-bank changes do not affect completed interviews.

Module	Design Description
Candidate Interface	Collects candidate details, resume, OTP verification input, selected role, camera permission, and interview answers.
Flask Application Layer	Controls routing, session data, interview start, API question loading, frame analysis, answer submission, and result display.
AI Evaluation Layer	Applies exact matching, keyword matching, TF-IDF cosine similarity, communication score, and feedback generation.
Computer Vision Layer	Receives webcam frames from the browser and detects face presence, centered face status, and multiple-face alerts using OpenCV.
Database Layer	Maintains roles, users, question bank, assigned interview questions, answers, scores, shortlist status, and report paths.
Report Layer	Generates a PDF interview report containing marks, feedback, confidence index, and shortlist decision for admin review.

4.5 Database Design and Question Bank Flow

The second-review implementation improves the data layer by using PostgreSQL as the live source of truth for the editable question bank. Each question contains role, section, topic, difficulty, options, correct answer, keywords, marks, and active status. When an interview starts, the system selects questions from the active question bank and records the assigned paper for that specific interview.

Table / Data Area	Purpose
roles	Stores role names such as Manual Testing, Automation Testing, AI Tech Support, and Software Development.
users	Stores candidate name, email, phone number, OTP status, resume path, and registration time.
question_bank	Stores editable role-wise aptitude and programming questions with answers, keywords, and usage count.
interviews	Stores interview id, candidate id, selected role, date, total score, status, shortlist status, and report path.
interview_questions	Stores the exact questions assigned to a candidate for audit and consistency.
candidate_answers	Stores submitted answers, AI score, feedback, matched keywords, and missing keywords.

4.6 80% Code Implementation Snapshots

The project has reached the intermediate implementation stage. The major working modules include candidate registration, OTP verification, role selection, PostgreSQL question bank, NLP answer evaluation, OpenCV camera monitoring, PDF report generation, and admin report viewing. The following code snapshots show the implemented logic used in the current system.

Snapshot 1: NLP Answer Scoring Logic

```
def score_answer(answer, question):
    if question.get("category") in {"Aptitude", "Programming"}:
        exact = exact_answer_score(answer, question["correct_answer"])
        return {"total_score": round(exact * 100, 2)}

    kw = keyword_score(answer, question.get("keywords", []))
    sem = semantic_score(answer, question.get("ideal_answer", ""))
    comm = communication_score(answer)
    total = (kw * 0.35) + (sem * 0.45) + (comm * 0.20)
```

Snapshot 2: PostgreSQL Question Bank Selection

```
def select_questions_for_role(role_slug, excluded_ids=None):
    selected = []
    with get_db() as conn:
        for section, count in SECTION_COUNTS.items():
            pool = conn.execute("""
                SELECT * FROM question_bank
                WHERE role_slug = %s AND section = %s AND active = TRUE
            """, (role_slug, section)).fetchall()
            pool.sort(key=assignment_sort_key)
            selected.extend(pool[:count])
```

Snapshot 3: OpenCV Camera Monitoring Flow

```
faces = face_cascade.detectMultiScale(gray, 1.2, 5, minSize=(60, 60))
if len(faces) > 0:
    x, y, w, h = max(faces, key=lambda r: r[2] * r[3])
    face_ratio = (w * h) / frame_area
    response = {
        "face_detected": True,
        "multiple_faces": len(faces) > 1,
        "confidence_hint": "Face detected and centered"
    }
```

CHAPTER 5

RESULTS OBTAINED (INTERMEDIATE)

The intermediate result of the AI-Based Smart Interview System shows that the main modules are working together as a complete interview-assessment workflow. The system can

register candidates, verify OTP, collect resume files, display role-based interview questions, evaluate answers, monitor face presence, store results, and generate reports for admin review.

Feature Tested	Intermediate Result
Candidate Registration and OTP	Candidate details are collected and OTP verification controls access to the interview flow.
Role Selection	Candidate can select Manual Testing, Automation Testing, AI Tech Support, or Software Development role.
Question Generation	Role-based aptitude and programming questions are loaded from the PostgreSQL question bank.
Answer Evaluation	Objective answers are checked exactly and descriptive answers are scored using keyword, TF-IDF, and communication scores.
Face Monitoring	Camera frames are analyzed for face presence and multiple-face warning conditions.
Report Generation	Final score, feedback, confidence index, shortlist status, and report path are generated for review.

5.1 Sample Intermediate Output

- ? The interview page displays questions, answer options, camera status, and submit controls.
- ? The backend calculates question-wise score and total candidate score after submission.
- ? The report module prepares a PDF report with candidate name, score, feedback, shortlist status, and confidence summary.
- ? The admin report page helps the evaluator view generated reports without manually checking every file.

5.2 Current Completion Status

Around 80% of the implementation is completed for the second review. The remaining work includes final UI polishing, larger question-bank validation, extended testing with more candidates, report-format refinement, and final documentation preparation.

CHAPTER 6

REFERENCES

- [1] I. Naim et al., "Automated Prediction and Analysis of Job Interview Performance," 2015.
- [2] L. Nguyen and D. Gatica-Perez, "Hirability in the Wild: Analysis of Online Video Resumes," 2016.
- [3] L. Chen et al., "Automated Video Interview Judgment on Large Corpus," 2017.
- [4] S. Muralidhar et al., "Words Worth: Verbal Content and Hirability Impressions," 2018.
- [5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," 2011.
- [6] K. Sparck Jones, "A Statistical Interpretation of Term Specificity," 1972.
- [7] G. Bradski, "The OpenCV Library," 2000.

- [8] J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers," 2019.
- [9] H. Suen et al., "Artificial Intelligence in Personnel Selection," 2019.
- [10] N. Guenole et al., "Algorithmic Recruitment and Selection Systems," 2022.
- [11] Python Software Foundation, Python Documentation.
- [12] Flask Documentation, Pallets Projects.
- [13] PostgreSQL Global Development Group, PostgreSQL Documentation.
- [14] ReportLab User Guide for PDF Generation.